

A Simple Calculator

Student ID: 12412308

Name: 李语尚

Course: CS219: Advanced Programming

Date: 2026 年 3 月 22 日

目录

1 前言	3
2 功能展示	3
2.1 运行环境	3
2.2 交互式输入输出	3
2.3 支持多种数字输入形式	3
2.4 支持部分函数调用	3
2.5 错误输入识别	4
3 功能实现	4
3.1 心路历程	4
3.2 数据存储方式	4
3.3 加减运算	5
3.3.1 原理	5
3.3.2 实现	5
3.4 乘法算法	5
3.4.1 原理	5
3.4.2 实现	6
3.5 除法算法	6
3.5.1 原理	6
3.5.2 实现	6
3.6 开方算法	7
3.6.1 原理	7
3.7 指数计算	7
4 一路走来有多不容易	7
4.1 心路历程 2	7
4.2 关于 AI 使用	8
4.3 C and java	8
4.4 期望	8

目录	2
5 结语	8
6 参考文献	8

1 前言

这个程序通过用数组模拟数据，并且通过了“压位”的方式，实现了高精度的数据计算。作为 C 语言的初学者，这个 project 的实现是让人心力憔悴的。并且我不得不承认如果没有 ai 的帮助，我是无法完成这个 project 的。这个报告呈现了我实现这个计算器的思路以及为什么要这么实现的思考，以及对 ai 使用的思考。

2 功能展示

2.1 运行环境

本程序是由 C 语言编写。在 Linux (Ubuntu / WSL) 系统中，使用 gcc 编译器进行了编译和运行。

```
$ gcc main.c -o main
$ ./main
```

2.2 交互式输入输出

程序支持交互式命令行模式，用户可以直接输入表达式，程序会即时输出结果。支持 ‘quit’ 命令退出。

```
> 1 + 1
1 + 1 = 2
> 100 / 3
100 / 3 = 33.333333333333333333
```

2.3 支持多种数字输入形式

输入解析器非常鲁棒，支持以下格式：

- 普通整数：123
- 小数：123.456
- 科学计数法：1e10, 1.23e-5, 1E+100
- 带有冗余空格的输入：1 + 2

2.4 支持部分函数调用

内置了开根、乘方函数，支持直接在计算中使用。调用方式为 ‘sqrt(x)’，‘x^n’。

```
> sqrt(10)
sqrt(10) = 3.162277660168379332
> 2 ^ 10
2 ^ 10 = 1024
> 3 ^ 100
3 ^ 100 = 515377520732011331036461129765621272702107522001
```

2.5 错误输入识别

对于非法输入（如除以零、格式错误的数字、不支持的运算符），程序会捕捉并打印友好的错误信息，而不会导致程序崩溃。

```
> 1 / 0
```

```
Error: 除数不能为零。
```

```
> 1 + abc
```

```
Error: 输入无法解释为数字。
```

```
>ysq is handsome
```

```
Error: 无效输入: 预期格式 '<数字> <操作符> <数字>'。
```

3 功能实现

3.1 心路历程

仅仅使用 C 语言内置的 `double` 或 `long double` 是不够的。我们需要自己管理数字的每一位。ai 发展的确实有点快，现在 ai 都不会建议用 `longlong` 了，而是提出了各种好方法来存储数据。我想，既然“计算”这件事情在哪里都用得到，不如就去找一下开源的 `postgres` 计算代码，看完一万两千行代码显然是不现实的，于是我把这一大坨代码丢给了 `copilot`，帮我介绍一下存储数据的结构，以及计算的实现。并且上学期数字逻辑的全加器还有乘法计算器也提供给了我一些思路。在经过可行性分析之后，我的思路如下：

3.2 数据存储方式

一个封装的数字 ‘`NumericVar`’ 共有以下关键信息：存储每一位的动态数组 `digits[]`，一个整数变量存储数组的长度 `ndigits`，一个整数变量存储相对于小数点的权重 `weight`，以及符号标志 `sign`。为了计算方便，我们把最高有效位 (MSB) 放在了数组的第一位 (`digits[0]`)。

我们的数值实质上可以表示为：

$$\text{Value} = \text{sign} \times \left(\sum_{i=0}^{\text{ndigits}-1} \text{digits}[i] \times \text{BASE}^{\text{weight}-i} \right)$$

其中 $\text{BASE} = 1000$ ， $\text{digits}[i] \in \{0, 1, \dots, \text{BASE} - 1\}$ 。说大白话，就是一千进制。

```
1 #define BASE 1000
2 typedef struct {
3     unsigned int *digits; // 动态数组，存储 BASE 进制的位
4     int ndigits;           // 数组长度
5     int weight;           // 权值：小数点相对于 digits[0] 的位置
6     int sign;             // 符号：1 正，-1 负，0 零
7     int scale;            // 显示精度
8 } NumericVar;
```

3.3 加减运算

3.3.1 原理

加减法的核心在于对齐小数点。由于我们使用 `weight` 来表示位权，两个数相加前，必须先把它们的指数 (`weight`) 对齐，通常是将精度较低的数对齐到精度较高的数 (即补零)。

3.3.2 实现

```

1  1. 如果 A.符号 != B.符号:
2      // 异号相加转为减法
3      如果 |A| > |B| 返回 SubAbs(A, B), 符号为 A.符号
4      否则 返回 SubAbs(B, A), 符号为 B.符号
5  2. 同号相加 (AddAbs)
6      初始化 R, 分配足够空间
7      设 R.权重 = max(A.权重, B.权重) + 1 // 预留进位空间
8      进位 carry = 0
9  3. // 从低位向高位遍历
10     对于 i 从 0 到 (max_weight - min_weight):
11         val_a = A 在当前权重位的数值 (若无则为0)
12         val_b = B 在当前权重位的数值 (若无则为0)
13         sum = val_a + val_b + carry
14         R.digits[i] = sum % BASE // 当前位
15         carry = sum / BASE // 计算进位
16 4. 去除 R 的前导零和无效零
17 5. 返回 R

```

```

1  // 伪代码逻辑: SubAbs(A, B) (前置条件 |A| >= |B|)
2  1. 初始化 R, R.权重 = A.权重
3     借位 borrow = 0
4  2. // 从低位向高位遍历
5     对于 i 从 0 到 R.长度:
6         val_a = A 在当前权重位的数值
7         val_b = B 在当前权重位的数值
8         diff = val_a - val_b - borrow
9         如果 diff < 0:
10             diff = diff + BASE // 向高位借位
11             borrow = 1
12         否则:
13             borrow = 0
14         R.digits[i] = diff
15 3. 去除 R 的前导零, 返回 R

```

3.4 乘法算法

3.4.1 原理

实现了经典的长乘法，时间复杂度为 $O(N \times M)$ 。

3.4.2 实现

```

1 // 伪代码逻辑: Multiply(A, B)
2 1. 初始化 R, 分配空间 = A.长度 + B.长度
3   R.digits 全部置零
4   R.权重 = A.权重 + B.权重 + 1
5 2. // 长乘法 (Long Multiplication)
6   对于 i 从 0 到 A.长度 - 1:
7       如果 A.digits[i] == 0: continue
8       进位 carry = 0
9       对于 j 从 0 到 B.长度 - 1:
10          // 计算在结果数组中的位置
11          k = i + j + 1
12          // 累加乘积: 当前位 = 旧值 + (A位 * B位) + 进位
13          sum = R.digits[k] + (A.digits[i] * B.digits[j]) + carry
14          R.digits[k] = sum % BASE
15          carry = sum / BASE
16       // 处理当前行剩余的进位
17       R.digits[i] += carry
18 3. R.符号 = A.符号 * B.符号
19 4. 去除 R 的前导零, 返回 R

```

3.5 除法算法

3.5.1 原理

实现了 Knuth 的 Algorithm D, 这是一种能够处理多精度整数除法的标准算法。

3.5.2 实现

```

1 // 伪代码逻辑: Divide(A, B) (Knuth Algorithm D)
2 1. 归一化: 确保 B 的最高位 >= BASE/2 (提高估商精度)
3   Q.权重 = A.权重 - B.权重
4   dividend = A (作为余数缓冲区)
5 2. // 主循环: 从高位向低位试商
6   对于 shift 从 Q.权重 递减到 0:
7       // 估算商 digit (q_hat)
8       // 使用 dividend 的最高2位 (u0, u1) 和 B 的最高1位 (v1)
9       q_hat = (u0 * BASE + u1) / v1
10      如果 q_hat >= BASE: q_hat = BASE - 1
11      // 修正 q_hat: 如果 (B * q_hat) > 当前对应的 dividend 片段
12      当 (B * q_hat) > current_dividend_window:
13          q_hat = q_hat - 1
14      // 记录商
15      Q.digits[对应下标] = q_hat
16      // 乘减操作: 更新 dividend (余数)
17      dividend = dividend - (B * q_hat * BASE^shift)
18 3. 返回 Q, dividend (此时即为 Rem)

```

3.6 开方算法

3.6.1 原理

采用以整数为基础的牛顿迭代法 (Newton-Raphson Method)。迭代公式：

$$x_{n+1} = \frac{1}{2}(x_n + \frac{a}{x_n})$$

为了处理小数，先将数字放大 10^{2k} 倍变成整数开方，最后再缩小。

```

1 // 伪代码逻辑: Sqrt(N, scale)
2 1. // 放大转换
3   M = N * 10^(2 * scale)
4 2. // 初始猜测 X_0
5   根据 M 的十进制位数估算, X_0 = 10^(digits(M) / 2)
6 3. // 牛顿迭代循环
7   重复直到收敛 (X_new >= X_old):
8       X_new = (X_old + M / X_old) / 2
9       更新 X_old = X_new
10 4. // 结果修正 (确保是 floor(sqrt(M)))
11   如果 (X+1)^2 <= M: X = X + 1
12   如果 X^2 > M: X = X - 1
13 5. // 缩小还原
14   结果 = X * 10^(-scale)
15   返回 结果

```

3.7 指数计算

```

1 // 伪代码逻辑: Power(Base, Exp)
2 1. R = 1
3 2. 循环 i 从 1 到 Exp:
4     R = Multiply(R, Base) // 调用长乘法
5 3. 返回 R

```

4 一路走来有多不容易

4.1 心路历程 2

刚开始和朋友们讨论这个 project 应该怎么实现的时候，我说我看很多程序都是压位实现的，感觉很高效，的确压位可以少计算很多，朋友劝我：“你一位存一个不就好了？为什么还要 BASE1000，这不是自找哭吃吗”，当时我不以为然，但是现在我敲下报告的时候，才知道这一路走来有多不容易。我选择压位错了吗？probably not. 课上学过，一个 int 占 4 个 bytes，课上让我体会到，C 语言与之前的 java 相比，的确更底层一些，甚至跟我这学期刚开始学的汇编有一些相似之处，这是之前的 java 从未带给我的感受。所以我怎么忍心让 32 个 bits 的前几十位都是空荡荡的 0 呢。postgres 的源码用的是 BASE10000，我怕不知道突然出现的 overflow 让我抓耳挠腮，于是退而求其次，选择了 BASE1000，希望在我看不见的内存里，bit 更多的是 1 而不是 0 吧。

4.2 关于 AI 使用

truth be told, 我大量的使用了 ai, 包括了以下场景: 帮我读源码, 帮我分析算法可行性, 在我有思路后, 帮我写代码, 帮我分析有没有野指针, 帮我分析内存有没有泄露, 计算结果不对的时候帮我 debug, 帮我写测试脚本查看我的答案是否正确... 这种感觉很像: 你去到从未去过的国家, 语言不通, 文化不通, 在你一个人手足无措的时候, 另一个人出现了, 她毫无保留的帮助你, 指针、地址、内存管理、什么时候用., 什么时候用->, 什么时候用 *, 什么时候用 &, 你无法抑制地爱上了她, 依赖着她... 但我知道, 只有做一个独立的人, 才是我想要的。在这个 project 中, 在 AI 的带领下, 我勉勉强强地完成了任务, 希望在后面的 project 中, “我”的地位会越来越强, 直到脱离依赖, 在 C 语言的世界里独立生存。

4.3 C and java

一年之前, 我第一次接触了编程, 学的是 java, JVM 给了我足够的安全感, 指针? 内存? 我通通不需要直到, 现在 C 语言带给我的感觉就像是成年之后失去了家长的庇护, 你必须正面你拥有的地址, 确保它的内容干净, 一次又一次检查, 清理。初识 C 语言的感觉并不友好, 就像脱离了自己的舒适区一样, 希望在后面的 project 中, 逐渐感受到它的强大。

4.4 期望

希望下一个 project 少用 ai, 多自己手搓, 有更多自己的思路, 多钻研, enjoy coding。

5 结语

通过本项目, 我不仅深入理解了高精度运算的底层原理 (位权管理、进位处理、数值分析算法), 更学会了如何高效地与 AI 协作。从代码生成到 Bug 修复, 再到自动化测试体系的构建, AI 工具极大地提升了工程效率, 让我能将更多精力集中在核心算法逻辑的设计上。

6 参考文献

参考文献

- [1] GitHub. (n.d.). GitHub Copilot
- [2] A Simple Calculator. Retrieved from <https://github.com/BrightonXX/SUSTech-CPP-Project.git/project1/calculator.c>
- [3] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.
- [4] The PostgreSQL Global Development Group. (n.d.). PostgreSQL Source Code (numeric.c). Retrieved from <https://github.com/postgres/postgres>